

IN422 : TP4 sur les graphes orientés

Modalités de rendu

Chaque binôme rendra :

1. une archive contenant le code réalisé ;
2. séparément de cette archive, un compte-rendu au format .pdf commentant les choix qui ont été faits, et contenant les analyses de vos expérimentations.

Il vous est demandé de produire vous-mêmes le code demandé, ainsi que le compte-rendu. Les emprunts aux collègues, et la délégation du travail à des IAs, ne vous permettront pas de récolter les bienfaits attendus : renforcer des savoirs, développer des savoir-faire. Dans le premier cas, il s'agit de plagiat, qui peut vous exposer à des sanctions.

Vous prendrez soin d'inclure vos deux patronymes dans chacun des deux fichiers. Les fichiers sont à déposer sur Chamilo, au plus 7 jours après la séance de TP.

Introduction

Dans ce TP, vous allez implémenter en Python des algorithmes sur les graphes du cours. Pour chaque partie deux fichiers sont fournis : <nom>.py et test_<nom>.py. Vous complétez ces fichiers, et jouerez les tests fournis ou vos propres tests avec par exemple la commande `python3 test_<nom>.py`. Vous pourrez (c'est même recommandé) vous inspirer des éléments vus dans le TD sur les graphes orientés (un corrigé du code demandé dans ce TD est sur Chamilo).

Vous allez utiliser des implémentations Python des TAD liste, file, pile et tas vus en cours et en TP précédemment. Vous noterez une différence essentielle entre les implémentations Python et les implémentations C des précédents TPs. Par exemple la fonction :

```
liste* ajoute(liste* lst, int i);
```

ne modifie pas la liste `lst` passée en paramètre. Elle crée et renvoie une nouvelle liste (qui partagera des éléments avec la liste passée en paramètre).

Au contraire dans les implémentations Python, tout se passe comme si les objets étaient "passés par référence", et ils sont modifiés s'ils ne sont pas explicitement recopiés :

```
lst = [0,2]
lst.append(1) # la valeur désignée par lst est maintenant [0,2,1]
```

1 Implémentation efficace de Dijkstra

Dans cette partie le but est d'implémenter l'algorithme de Dijkstra pour des graphes valués encodés en Python comme illustré dans l'exemple suivant :

```
gv = {"a": [("f", 14), ("c", 9), ("b", 7)], \
      "b": [("c", 10), ("d", 15)], \
      "c": [("f", 2), ("d", 11)], \
      "d": [("e", 6)], \
      "e": [], \
      "f": [("e", 9)]}
```

Pour le graphe valué (G, ℓ) encodé par `gv`, la longueur de l'arc (a, f) est 14. Dans la suite, `gv` désignera un dictionnaire encodant un graphe valué (G, ℓ) comme dans cet exemple. Les fichiers utiles dans cette partie sont `dijkstra.py` et `test_dijkstra.py` qui sont à compléter avec du code Python, et `gv_valence.py` et `graphe_valence.json` qui permettent de désérialiser un graphe valué des rues de Valence.

1. Coder et tester la fonction `longueur_chemin(gv, c)` qui prend en entrée `gv` et une liste `c` de sommets de G , et renvoie la longueur $\ell(c)$ si la liste `c` encode un chemin dans G , et INFINI sinon.
2. Codez une fonction `dijkstra(gv, s)` qui prend en entrée `gv` et un sommet `s` de G , et qui renvoie un couple (d, p) où `d` est un dictionnaire encodant la fonction longueur d'un plus court chemin depuis `s`, et `p` est un dictionnaire qui encode la fonction prédécesseur permettant de reconstruire les plus courts chemins. Dans cette première implémentation, un arc sortant optimal sera déterminé à chaque itération, sans tenter de conserver les résultats des calculs d'une itération sur l'autre.
3. Nous allons améliorer cette première version. L'idée est de conserver les arcs sortants indexés par la longueur d'un possible plus court chemin correspondant dans une file de priorité. Ce TAD permet d'accéder aux éléments d'une collection en fonction d'un critère donnant leur priorité. Les files de priorités sont implémentées efficacement en Python à l'aide de tas binaires, sur la base des tableaux dynamiques que sont les listes Python. Voici un exemple élémentaire de mise en œuvre en Python :

```
from heapq import heappush, heappop
fp = [] # les listes Python sont des tableaux dynamiques,
        # comme dans le TP3 c'est une bonne base pour un tas min
heappush(fp, (0, "Alice")) # rajoute "Alice" avec priorité 0
heappush(fp, (1, "Bob")) # rajoute "Bob" avec la priorité 1
p,e = heappop(fp) # renvoie 0, "Alice", et retire ce couple de fp
```

On pourra avec une telle structure de donnée ajouter un nouvel arc sortant en $O(\log |A|)$, accéder à un arc sortant optimum en $O(1)$ et le retirer en $O(\log |A|)$.

Codez une fonction `dijkstra_optimise(gv,s)` avec les mêmes entrées et les mêmes sorties que `dijkstra(gv,s)`, et une complexité en $(|S| + |A|) \log |A|$.

4. Cette question est facultative. Application. Le graphe routier de la commune de Valence (tel qu'il peut être obtenu avec Open Street Map) est désérialisé en appelant la fonction `gv_valence()` du fichier `gv_valence.py`, qui renvoie un couple. La première composante est un graphe valué, comportant environ 2000 sommets identifiés par des entiers. La deuxième composante est un dictionnaire dont les clés sont les entiers identifiant les sommets du graphe, et les valeurs des textes décrivant ces sommets, avec des coordonnées géographiques que vous pourrez utiliser telles quelles sur Google Map.

Déterminez, en utilisant des appels à la fonction `dijkstra_optimise()`, deux sommets du graphe renvoyé par `gv_valence()` qui réalisent le diamètre du graphe, c'est à dire la plus grande distance entre deux sommets.

2 Existence de circuit dans un graphe orienté

Nous quittons les graphes orientés valués pour revenir aux graphes orientés. On s'intéresse à présent au problème de décision suivant : étant donné un graphe orienté G , existe-t-il un circuit dans G ? Le but de cette partie est de résoudre par un algorithme ce problème de décision. L'algorithme donnera aussi un certificat de la réponse donnée, qui sera utilisé par votre code de test pour valider votre implémentation.

5. Codez et testez la fonction `est_tri_topologique(g,t)`. Cette fonction prend en entrée un premier dictionnaire g encodant un graphe orienté et un second dictionnaire t encodant une fonction définie sur l'ensemble des sommets du graphe et à valeurs réelles. Elle renvoie `True` si et seulement si cette fonction est un tri topologique sur le graphe.
6. Codez et testez la fonction `tri_topologique(g)` vue en cours. Elle prend en entrée un graphe g et renvoie un dictionnaire t implémentant un tri topologique sur les sommets de g lorsque ce graphe orienté est sans circuit. Elle utilise un parcours en profondeur itératif pour affecter des valeurs décroissantes aux sommets au fur et à mesure qu'ils sont dépilés. On utilisera le fait que les listes Python implémentent directement et efficacement le TAD pile, le sommet de la pile correspondant à l'élément de plus grand indice.
7. Codez et testez la fonction `est_circuit(g,c)`, qui renvoie `True` si et seulement si la liste c encode un circuit du graphe encodé par le dictionnaire g .
8. Codez et testez la fonction `decision_circuit(g)` qui renvoie :
 - un couple (True, c) avec c encodant un circuit lorsque le graphe encodé par g comporte au moins un circuit ;
 - un couple (False, t) avec t qui encode un tri topologique lorsque le graphe encodé par g ne comporte pas de circuit.

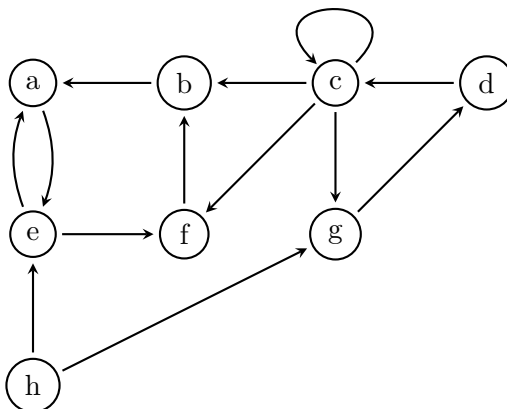
On pourra partir de la base fournie par le code de `tri_topologique`, c'est à dire essentiellement un parcours en profondeur itératif, et utiliser les deux observations suivantes :

1. à tout moment du parcours, la liste implémentant la pile est un chemin du graphe,
2. si le graphe encodé par g possède un circuit, alors il existe un sommet du graphe qui sera dans la pile quand un de ses prédécesseurs sera au sommet de la pile.

3 Calcul des composantes fortement connexes d'un graphe orienté

3.1 Compléments théoriques sur les composantes fortement connexes

Dans un graphe orienté $G = (S, A)$, comme par exemple celui-ci :



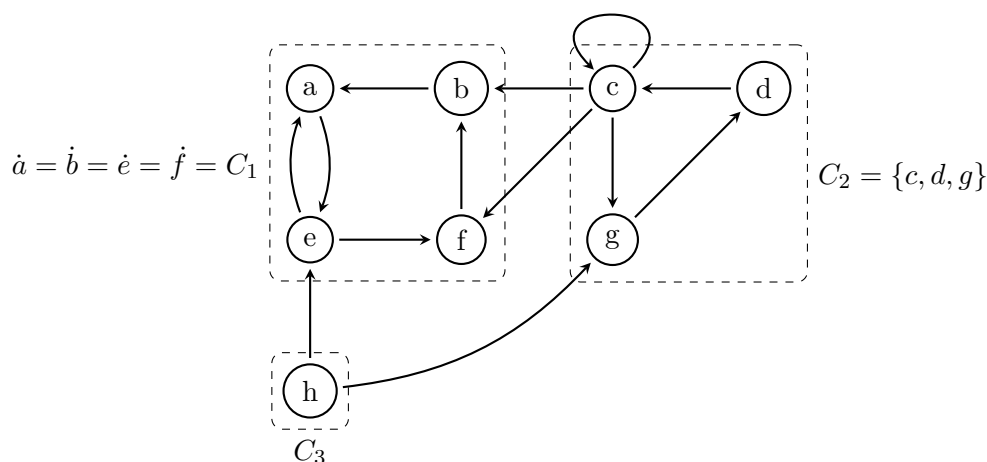
on peut considérer la relation $\mathcal{R}$ suivante sur l'ensemble des sommets S :

$$\forall x, y \in S, x\mathcal{R}y \Leftrightarrow x \text{ est accessible depuis } y \text{ et } y \text{ est accessible depuis } x$$

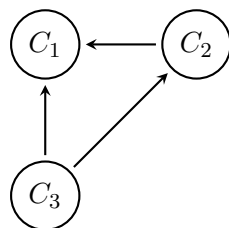
Cette relation est réflexive puisque (x) est un chemin de x à x , symétrique par construction, et transitive comme on le vérifie facilement. C'est donc une relation d'équivalence sur S , et ses classes d'équivalence sont appelées les composantes fortement connexes de G :

$$\dot{x} = \{y \in S \mid y\mathcal{R}x\} \text{ est la composante fortement connexe de } x \text{ dans } G$$

Les trois composantes fortement connexes du graphe précédemment donné en exemple sont représentées ci-après :



Les composantes fortement connexes de G forment une partition de S , et l'ensemble $S/\mathcal{R}$ de ces composantes est naturellement un graphe orienté en considérant que pour $x, y \in S$, le couple $(\dot{x}, \dot{y})$ est un arc si et seulement si $\dot{x} \neq \dot{y}$ et qu'il existe un arc $(x_0, y_0) \in A$ avec $x_0 \in \dot{x}$ et $y_0 \in \dot{y}$. Ce graphe est appelé le graphe quotient de G , et il est représenté ci-dessous pour notre graphe d'exemple :



On constate que pour notre exemple le graphe quotient est sans circuit, et on vérifie facilement que c'est un fait général.

3.2 Calcul des composantes fortement connexes sur un exemple

Pour comprendre l'algorithme de calcul des composantes fortement connexes à implémenter dans la partie pratique, nous allons travailler sur l'exemple donné dans la partie théorique. Les questions de cette sous-partie sont directes et ne nécessitent pas de produire du code.

9. Vérifiez que C_1 est un puits de $G/\mathcal{R}$. Que renvoie $\text{ACCESSIBLES}(G, a)$?
Que renvoie $\text{ACCESSIBLES}(G, x)$ lorsque $x \in C_1$?
10. Vérifiez que dans $G' = G[S \setminus C_1]/\mathcal{R} = G/\mathcal{R}[\{C_2, C_3\}]$, le graphe quotient induit par les sommets non dans C_1 , le sommet C_2 est un puits. Que renvoie $\text{ACCESSIBLES}(G[S \setminus C_1], x)$ lorsque $x \in C_2$?

Les deux questions précédentes illustrent un premier fait essentiel dans l'algorithme à implémenter dans la partie pratique : **si accessible est lancé dans un sous-graphe induit depuis un sommet situé dans un puits du graphe quotient, alors il renvoie exactement une composante fortement connexe. On peut alors retirer les sommets de cette composante, et repartir d'un nouveau puits dans le sous-graphe induit résiduel.**

Il reste à déterminer un ordre sur les sommets de manière à toujours partir d'une composante puits. Pour cela, c'est à nouveau l'ordre de fin de visite dans un parcours en profondeur qui va nous être utile.

11. On observe que (C_2, C_1) est un arc de $G/\mathcal{R}$. Si on effectue un parcours en profondeur de G en démarrant d'un sommet de C_1 , par exemple le sommet a , pourquoi tous les sommets de C_2 sortiront de la pile après tous les sommets de C_1 ?
12. Si on effectue un parcours en profondeur en partant d'un sommet de C_2 , par exemple c , pourquoi ce sommet sortira de la pile après tous les sommets de C_1 ?

Ces deux questions illustrent le deuxième fait essentiel dans l'algorithme à implémenter dans la partie pratique, que nous énonçons à présent. **Soit $\dot{x}$ et $\dot{y}$ deux composantes telles que $(\dot{x}, \dot{y})$ soit un arc du quotient. Alors le sommet de $\dot{x}$ dont la visite a débuté en premier sera toujours sorti de la pile après tous les autres sommets de $\dot{x}$ et après tous les sommets de $\dot{y}$. On en déduit que le sommet sorti en dernier de la pile est dans une source du quotient.** C'est le cas du sommet h dans notre exemple. **Pour obtenir des sommets dans des puits du quotient, il suffira donc de lancer le parcours en profondeur sur le miroir de G .**

Si on récapitule, l'algorithme à implémenter dans la partie pratique se déroule de la manière suivante sur notre exemple :

1. On commence par un parcours en profondeur du miroir G qui renvoie la liste des sommets dans un ordre qui permettra d'obtenir les puits successifs du miroir quotient induit : en premier, le sommet sorti en dernier de la pile. Par exemple l'ordre (a, b, f, e, c, d, g, h) .
2. On parcourt cette liste dans l'ordre :
 - D'abord a , on le groupe avec ses accessibles dans G , pour obtenir une première composante connexe $\{a, b, e, f\}$, et on poursuit dans le graphe induit par les autres sommets,
 - ensuite c , que l'on groupe avec ses accessibles pour obtenir la composante $\{c, g, d\}$,
 - enfin h et on obtient $\{h\}$.

3.3 Partie pratique

13. Codez et testez la fonction `dfs(g)` qui renvoie la liste des sommets dans l'ordre postfixe inverse, c'est à dire par dates de sortie de la pile décroissantes.
14. Implémentez efficacement la fonction `accessibles(g, s, f)` qui renvoie l'ensemble des sommets accessibles depuis s dans $G[F]$, le sous-graphe de G induit par l'ensemble $F \subset S$, étant donné que g encode un graphe $G = (S, A)$ et que f encode la fonction indicatrice de F .
15. Implémentez efficacement la fonction `cfc(g)` qui renvoie les composantes fortement connexes du graphe encodé par g . On renverra un dictionnaire encodant la fonction qui à chaque sommet fait correspondre sa composante fortement connexe, identifiée par un entier. L'idée est d'itérer sur les sommets dans l'ordre donné par un appel à `dfs(miroir(g))`, pour regrouper dans une même composante fortement connexe les sommets accessibles dans le sous-graphe induit par les sommets non encore dans une composante déjà calculée.
16. Cette question est facultative. Codez et testez la fonction `quotient(g)` qui renvoie le graphe quotient du graphe encodé par g .